ADVANCED PROGRAMMING LANGUAGES

Fall 2024, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Type-classes in Scala

In our last lecture we have discussed type-classes at work in Haskell. Here we will show how they can be implemented in Scala. We look at two approaches.

- The first is a direct transposition of the translation of type-classes as dictionaries, adopted in the Haskell implementation.
- The second relies on the suite of Scala's mechanisms for *contextual abstraction* to render Haskell's type classes and qualified types at a comparable level of abstraction.

Type classes as Traits, Class instances as case classes

The first, simple approach is based on a programming pattern that uses traits for a straightforward representation of the dictionaries employed in Haskell's translation-based implementation. We illustrate the pattern on the ordered insert function we introduce in our last lecture.

```
// does not compile, as <= is not a polymorphic operator
def insert[T](x:T, xs:List[T]) : List[T] =
          if xs.isEmpty || x <= xs.head then x::xs
    else xs.head :: insert(x, xs.tail)
```

We define a trait representing the family of types which admit an ordering relation, and package the ordering operators with the trait.

```
trait Ord[T]:
    def compare(x: T, y: T): Int
    def lteq(x:T, y:T) : Boolean = compare(x,y) <= 0
    def lt(x:T, y:T) : Boolean = compare(x,y) < 0
    def gteq(x:T, y:T) : Boolean = compare(x,y) >= 0
    def gt(x:T, y:T) : Boolean = compare(x,y) > 0
```

The `Ord` trait is essentially a duplicate of the `Ordered` trait provided by the standard library. As in that case, the ordering operators are defined in terms the `compare` function which is left abstract, as it depends on the instances of the type-class. The trait type-class is nevertheless useful, as it allows us to define `insert`, as follows:

```
def insert[T](x:T, xs:List[T])(ord:Ord[T]) : List[T] =
          if xs.isEmpty || ord.lteq(x, xs.head) then x::xs
     else xs.head :: insert(x, xs.tail)(ord)
```

The new definition uses the `ord:Ord[T]` object as an abstract dictionary to extract the implementation of the (overloaded) operator to be used for the desired comparison at the types of the family. Notice that the additional parameter is passed in the recursive call for further use as the list is examined.

Next, we create an instance of the dictionary for each type supporting the overloaded operations, and provide the corresponding implementation of the `compare` function.

```
case object IntOrd extends Ord[Int]:
          def compare(x:Int, y:Int) = x-y

case object DoubleOrd extends Ord[Double]:
          def compare(x:Double, y:Double) =
                    if (x == y) then 0 else if (x>y) then 1 else -1

case object StringOrd extends Ord[String]:
          def compare(x:String, y:String) = x.compare(y)
```

Finally, we can rewrite each call to the original `insert` function by passing the appropriate instance of `Ord[T]` as the argument for the dictionary:

```
insert(7, List(1,2,4,5))(IntOrd)
insert(5.6, List(3.2, 4.2, 1.3))(DoubleOrd)
insert("aa", List("aaa", "aab", "abc"))(StringOrd)
```

Contextual Abstractions

The simple approach we just illustrated is computationally effective, but somewhat low-level.

Next, we develop an alternative approach which is still based on a programming pattern that uses traits as type classes, but relies the language support for powerful mechanisms of *contextual abstraction* to express type classes at the same level of abstraction as Haskell. We start our discussion with an illustration of such mechanisms.

Givens

Often the behavior of an expression (or more in general, of a block of code) depends on the context in which it is evaluated (executed). The context may define data, i.e. system properties, permissions, timeouts, . . . etc. relevant to the expression's evaluation, or else define behavior, i.e. different implementations of operators such as equality.

Givens provide a general mechanism to introduce (system or user-defined) contextual data and behavior, and to access them automatically without having to make them explicit. There are several applications of this mechanisms: we look at the one directly relevant to type classes.

Type-class instances. Givens provide an elegant mechanism to introduce type class instances, as shown below. Given the `Ord[T]` trait introduced above as a type class, its (anonymous) instances may be defined as follows:

```
object Ord:
  given Ord[Int] with
    def compare(x: Int, y: Int) =
      if x < y then -1 else if x > y then +1 else 0

  given Ord[Double] with
         def compare(x:Double, y:Double) =
           if (x == y) then 0 else if (x>y) then 1 else -1

  given Ord[String] with
    def compare(x:String, y:String) = x.compare(y)
```

Givens are most often used in conjunction with parametrized types in the programming pattern for type classes. Back to our `insert` example, assuming the previous declaration of the `Ord[T]` trait, we can refactor our code as follows:

```
def insert[T](x:T, xs:List[T])(using ord:Ord[T]) : List[T] =
         if xs.isEmpty || ord.lteq(x, xs.head) then x::xs
    else xs.head :: insert(x, xs.tail)
```

The `ord:Ord[T]` parameter is now tagged with `using`, signaling its nature as a parameter which will be supplied by the context. Note that we don't need to supply it to the recursive call, as the compiler will fill it in automatically.

Having defined the givens in the `Ord` companion object, the calls to `insert` need not supply the `Ord[T]` argument, as the compiler will fill it in based on the type expected at the call site (thus providing for overloading resolution).

```
insert(-1, List(1,2,4,5))
insert(2.3, List(3.2, 4.2, 1.3))
insert("aa", List("aaa", "aab", "abc"))
```

Anonymous `using` clauses

In many situations, the name of a context parameter need not be mentioned explicitly in a `using` clause at all. In that case one can avoid defining a parameter name and just provide its type as in

```
def insert[T](x:T, xs:List[T])(using Ord[T]) : List[T] = . . .
```

We show two techniques to accomplish that.

`Summoned parameters. The first technique relies on the following method defined in Scala's standard library:

```
def summon[T](using t:T) = t
```

The effect of calling `summon[SomeType]` is that the compiler will look for a *given* definition of type `SomeType` and return that object as a result of a call to `summon[SomeType]`. Relying on this mechanism, we may the redefine our `insert` method with an anonymous context parameter of type `Ord[T]`, as follows:

```
def insert[T](x:T, xs:List[T])(using Ord[T]) : List[T] =
          if xs.isEmpty || summon[Ord[T]].lteq(x, xs.head) then x::xs
     else xs.head :: insert(x, xs.tail)
```

The missing name of the `using` clause is synthesized by the call to `summon[Ord[T]]` which retrieves the `Ord[T]` parameter providing the method `lteq`.

Refactoring dictionaries as method suites. Context parameters may be anonymized also as a result of an alternative approach to the implementation of type classes. We illustrate the approach on our running example of the trait type-class `Ord[T]`.

The idea is to refactor the trait from a pure dictionary collecting the ordering operators on values of type `T`, as it has been defined so far, to include also a method suite for values of type `T` belonging to the type-class. This can be accomplished very elegantly, by defining an extension for the type `T`:

```
trait Ord[T]:
  def compare(x: T, y: T): Int
  def lteq(x:T, y:T) : Boolean = compare(x,y) <= 0
  def lt(x:T, y:T)   : Boolean = compare(x,y) < 0
  def gteq(x:T, y:T) : Boolean = compare(x,y) >= 0
  def gt(x:T, y:T)   : Boolean = compare(x,y) > 0

  extension (l:T)
     def <  (r:T) : Boolean = lt(l,r)
     def <= (r:T) : Boolean = lteq(l,r)
     def >  (r:T) : Boolean = gt(l,r)
     def >= (r:T) : Boolean = gteq(l,r)
```

Now, to compare two objects of type `T` we have two alternatives: either invoke the desired comparison function of the dictionary on the two objects, as in `ord.lteq(x,y)`, or else invoke the corresponding operator directly on one of the two objects, as in `x.<= y` (or equivalently `x <= y`). With this latter alternative, the given instance is still needed to provide the required operator implementations, but needs not be mentioned explicitly, as mentioning the type is enough:

```
def insert[T](x:T, xs:List[T])(using Ord[T]) : List[T] =
        if xs.isEmpty || x <= xs.head then x::xs
        else xs.head :: insert(x, xs.tail)
```

Context bounds

A context bound is a shorthand for expressing anonymous context parameters described by types. Keeping with our running example, our `insert` function can be written with a context bound as follows:

```
def insert[T: Ord](x:T, xs:List[T]) : List[T] =
   ...
```

A bound like `: Ord` on a type parameter `T` of a method or class indicates a context parameter `using Ord[T]`.

Context bounds can be combined with subtype bounds. If both are present, subtype bounds come first, e.g.

```
def g[T <: B : C](x: T): R = ...
```

Context bounds provide a useful notation tool in a number of situations, two of which we discuss below.

Givens and type class instances for algebraic datatypes Keeping with our `Ord[T]`, we can defined instances of the class for standard and user-defined algebraic data types as in the following definition of ordering on lists.

```
given [T:Ord] : Ord[List[T]] with
         def compare(xs: List[T], ys: List[T]): Int = (xs, ys) match
                  case (Nil, Nil) => 0
                  case (Nil, _) => -1
                  case (_, Nil) => +1
                  case (x :: xs1, y :: ys1) =>
                  val  fst = summon[Ord[T]].compare(x, y)
                           if fst != 0  then fst else compare(xs1, ys1)
```

Here we have a given for `Ord[List[T]]` for all types `T` that belong to the `Ord` type class . Notice the use of `summon[Ord[T]]` in the code, to provide the anonymous instance of `Ord[T]` on which to invoke the `compare` method.

Type classes and qualified types: Haskell vs Scala

By now we have introduced all the mechanism that concur to the Scala type-class programming pattern. We review and summarize the different steps walking backward on the path traced so far , and drawing a parallel with the features we have illustrated for the Haskell.

Qualified types vs Context bounds Context bounds are essentially syntactic sugar but provide a neat representation of Haskell's qualified types in Scala. If we look at the final definition of our `insert` function, we can represent its type as the constrained polymorphic Scala type

```
[T:Ord] (T => List[T] => List[T])
```

This type, in turn, can be viewed as a faithful representation of the Haskell qualified type:

```
Ord a => a -> [a] -> [a]
```

We remark that context bounds express properties over type parameters that are orthogonal to the subtype relationships established by lower and upper bounds. When we write `[T <: Ordered[T]]` we are saying that `T` is a subtype of `Ordered[T]`. Instead, writing `[T : Ord]` we are not saying much about what `T` is, except that there is some notion of ordering associated with `T`.

Type classes vs parametric traits. Parametric traits are a straightforward representation of Haskell's type classes, that may include the default implementation of some of some functions and operators. Furthermore, they may additionally include a method suite for the members of the type-class, using `extension` methods, so as to enable method invocations on the values of the member types, as paradigmatic in object-oriented languages.

Givens vs type-class instances. Givens, in turn, provide a direct implementation device for creating type-class instances, and Scala's term inference mechanisms based on `using` clauses, in turn, enable a very effective mechanism à la Haskell, for automatically supplying them when required at the functions call sites.

A note on uniqueness of instances

A subtle but important difference between Haskell and Scala concerns *uniqueness of instances*. In Haskell, the language guarantees that there is at most one instance per pair (class, type): two `instance Num Int` declarations in the same program are a compile-time error. This is what makes the dictionary-passing translation well-defined — the compiler always knows which dictionary to insert.

Scala makes a different choice. Multiple given instances for the same type can coexist, provided they are not simultaneously in scope at the same call site. This permits *orphan instances* — a given for `Ord[T]` defined neither in `T`'s module nor in `Ord`'s — at the price of making instance selection sensitive to imports. The compiler will report an ambiguity error if two competing givens are visible at a call site, leaving the programmer to disambiguate. This greater flexibility is the practical companion of retroactive extensibility: orphan instances are how a third party can make a library type orderable without touching either the type or the class.

References and Credits

The material for this lecture is based on Chapters 21, 22 and 23 of the course textbook, Programming in Scala, as well as on material found on the online Scala documentation available at the Scala 3

reference website.